Universidade Cruzeiro do Sul
Ciência da Computação

Aula 12 - Paradigma Imperativo – Parte 3

Disciplina: Paradigmas de Linguagens de Programação
Professor: Álvaro Colombero

Objetivos



- Revisar o que são subprogramas e sua utilidade na programação de computadores.
- Conhecer os diferentes tipos de subprogramas.
- Conceituar parâmetros e suas diferentes formas de passagem.
- Determinar a forma de passagem de parâmetros mais adequada para cada situação.

Introdução

- A abstração procedural está embasada no conceito de subprogramas.
- Subprogramas permitem reduzir o consumo de memória e o tempo de codificação, além de facilitar a organização e a compreensão do código.
- O cabeçalho de um subprograma define o tipo deste subprograma, seu nome e a lista de parâmetros.
- Tipos de subprogramas:
  - Procedimentos – não retornam valores.
  - Funções – retornam valores.

Subprogramas

- Subprogramas são blocos de código que executam uma tarefa e possuem as seguintes características:
  - Tem um único ponto de entrada.
  - A unidade chamadora fica suspensa durante a execução do subprograma.
  - O controle retorna à unidade chamadora quando o subprograma é terminado.
  - A chamada a um subprograma é a requisição para que ele seja executado.

Subprogramas

Subprogramas que não retornam valores (procedimento):

//C++

```
void hipotenusaP(double cateto1, double cateto2) {
    cout << sqrt(cateto1 * cateto1 + cateto2 * cateto2);
}
```

//Pascal

```
procedure hipotenusaP(cateto1, cateto2: real);
begin
  writeln(sqrt(cateto1 * cateto1 + cateto2 * cateto2));
end;
```

Subprogramas

Subprogramas que retornam valores (funções):

//C++

```
double hipotenusaF(double cateto1, double cateto2) {
    return sqrt(cateto1 * cateto1 + cateto2 * cateto2);
}
```

//Pascal

```
function hipotenusaF(cateto1, cateto2: real): real;
begin
 hipotenusaF :=sqrt(cateto1*cateto1 + cateto2*cateto2);
end;
```

Subprogramas

- Existem duas maneiras dos subprogramas acessarem os dados que devem ser processados:
  - Acesso a variáveis não locais (declaradas em outros locais, mas visíveis ao programa);
  - Passagem de parâmetros.
- A primeira alternativa reduz a confiabilidade do programa.

Parâmetros

- **Parâmetro** é um identificador que aparece na declaração de um subprograma.
  - Parâmetro formal (ou apenas parâmetro) é o parâmetro definido na declaração do subprograma;
  - Parâmetro real (ou argumento) é o parâmetro que aparece na chamada do subprograma.
- Geralmente a associação entre os parâmetros é feita pela sua posição.
- Algumas linguagens permitem a atribuição de valores padrão aos parâmetros, os quais devem ser os últimos da listagem.

```cpp
#include <iostream>

int elevado(int x, int y = 2) {
  int t = 1;
  for (int i = 0; i < y; i++) {
    t = t * x;
  }
  return t;      }

int main() {
  int base, expo, res1, res2;
  std::cout << "Digite o valor da base: ";
  std::cin >> base;
  std::cout << "Digite o valor do expoente: ";
  std::cin >> expo;
  res1 = elevado(base, expo);
  res2 = elevado(base);
  std::cout << base << " elevado a " << expo << ": " << res1 << "\n";
  std::cout << base << " elevado ao quadrado: " << res2 << "\n";
  return 0;       }
```

Parâmetros

- Algumas linguagens permitem o uso de parâmetros nomeados. Neste caso a definição dos valores de cada parâmetro é feita pelo seu nome e não pela sua posição.
- Vantagens de usar parâmetros nomeados:
  - Manutenção e legibilidade de código, principalmente em funções com muitos parâmetros ou com parâmetros opcionais, pois demonstra claramente o que é passado para cada parâmetro.
- Exemplos de linguagens que suportam este tipo de parâmetro:
  - Python, C#, Kotlin, Ruby e Swift.

Parâmetros

```python
def mostrar_usuario(nome, idade, cidade):
    print("Nome:", nome)
    print("Idade: ",idade)
    print("Cidade: ",cidade)

# Utilizando parâmetros posicionais
mostrar_usuario("Maria", 25,"Lages")

# Utilizando parâmetros nomeados 1
mostrar_usuario(nome="João", idade=30, cidade="Florianópolis")

# Utilizando parâmetros nomeados 2
mostrar_usuario(cidade="Florianópolis", nome="João", idade=30)
```

Parâmetros

MLPClassifier




```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100,),
activation='relu', *, solver='adam', alpha=0.0001, batch_size='auto',
learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200,
shuffle=True, random_state=None, tol=0.0001, verbose=False, warm_start=False,
momentum=0.9, nesterovs_momentum=True, early_stopping=False, validation_fraction=0.1,
beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000)
```

[source]

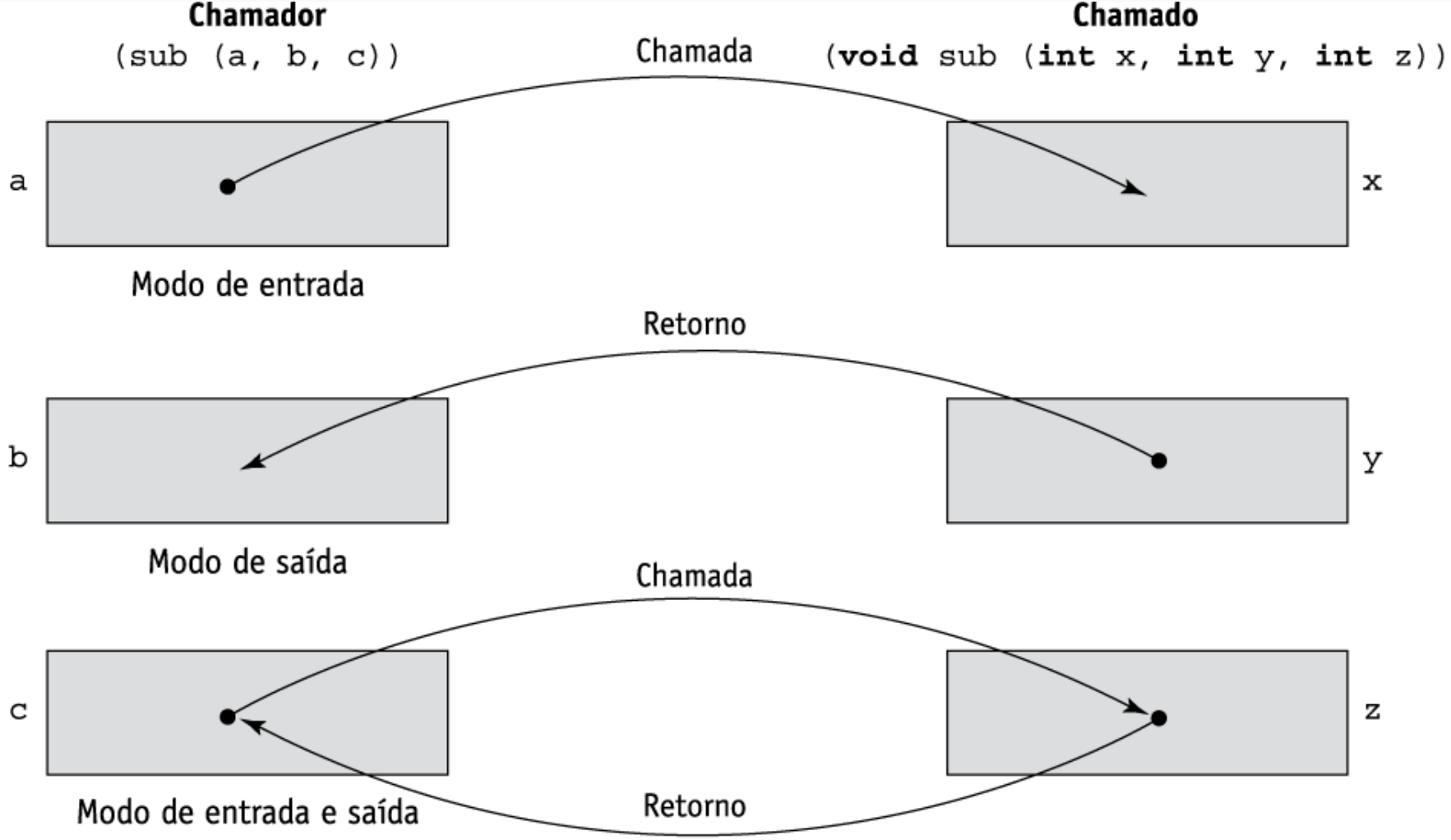


Figura 9.1 Os três modelos semânticos de passagem de parâmetros quando movimentações físicas são usadas.

Parâmetros

- Passagem por valor:
  - O valor do parâmetro real é passado para o parâmetro formal (entrada).
  - Não há retorno do parâmetro formal para o real (saída).
  - O parâmetro formal se comporta como uma variável local, que é inicializado com o valor do parâmetro real.
  - Normalmente realiza uma cópia do valor, o que demanda mais espaço de armazenamento.
  - Rápido para parâmetros pequenos mas pode ser lento em dados como matrizes.
  - Java suporta apenas passagem por valor para tipos primitivos.

Parâmetros

- Passagem por resultado:
  - O parâmetro formal se comporta como uma variável local que **NÃO** é inicializada automaticamente.
  - O valor do parâmetro formal é passado para o parâmetro real (saída).
  - Normalmente realiza uma cópia do valor, o que demanda mais espaço de armazenamento.
  - Rápido para parâmetros pequenos mas pode ser lento em dados como matrizes.
  - C# implementa com o modificador out, porém usa referência.

Parâmetros

- Passagem por valor-resultado:
  - O valor do parâmetro real é passado para o parâmetro formal (entrada).
  - O parâmetro formal se comporta como uma variável local, que é inicializado com o valor do parâmetro real.
  - O valor do parâmetro formal é passado para o parâmetro real (saída);
  - Normalmente realiza uma cópia do valor, o que demanda mais espaço de armazenamento.
  - Rápido para parâmetros pequenos mas pode ser lento em dados como matrizes.

Parâmetros

- Passagem por referência:
  - Abordagem diferente para implementar o modelo de entrada e saída.
  - Em vez de copiar os valores de uma lado para outro, o parâmetro oferece um caminho (endereço) de acesso ao dado existente no parâmetro real.
  - Não consome memória extra e a passagem é rápida.
  - O acesso aos dados é mais lento devido ao nível de indireção.
  - Mudanças inadvertidas causam mudanças que podem comprometer o resultado de outras partes do programa.

A maioria das linguagens de programação realiza, por padrão, a passagem por valor e possui mecanismos que permitem a passagem por referência.

```
using System;
class Program {
  static void Metodo1(int x) {
    x = x + 1;
  }

  static void Metodo2(ref int x) {
    x = x + 1;
  }

  public static void Main (string[] args) {
    int numero = 0;
    Console.WriteLine(numero);
    Metodo1(numero);
    Console.WriteLine(numero);
    Metodo2(ref numero);
    Console.WriteLine(numero);
  }
}
```

Parâmetros

- O C# apresenta diferentes modificadores para acessos aos parâmetros:
  - ref: o argumento deve ser inicializado antes de chamar o método. O método pode ou não atribuir um novo valor ao parâmetro.
  - out: o argumento não precisa ser inicializado antes de chamar o método. O método deve atribuir um novo valor ao parâmetro.
  - ref readonly: o argumento deve ser inicializado antes de chamar o método. O método não pode atribuir um novo valor ao parâmetro.
  - in: o argumento deve ser inicializado antes de chamar o método. O método não pode atribuir um novo valor ao parâmetro.
- Maiores detalhes em: https://learn.microsoft.com/pt-br/dotnet/csharp/language-reference/keywords/method-parameters

Paradigma Imperativo

Recapitulando…

- As linguagens imperativas são orientadas a ações e a computação é vista como uma sequência de instruções que manipulam valores de variáveis.
- Sua criação foi influenciada pela arquitetura de computadores *Von Neumann*, onde programas e dados são armazenados na mesma memória.
- São caracterizadas por 4 conceitos principais: Variáveis, Atribuição, Decisão e Repetição.
- Aspectos da programação imperativa prevalecem na maioria das linguagens de programação, sendo usados também nas linguagens do paradigma orientado a objetos.

Atividade 1



- Crie um programa em C# com três funções do tipo void, a primeira deve ler dois números inteiros, a segunda deve inverter a ordem dos dois valores e a terceira deve imprimi-los.
- Todo parâmetro deve possuir algum modificador.
- Descreva a forma de passagem de parâmetros utilizada em cada função.

Bibliografia

- Material complementar:

TUCKER, A. B.; NOONAN, R. E.; **Linguagens de programação: princípios e paradigmas.** 2. ed. São Paulo: McGrall-Hill, 2009. Seções 9.1 a 9.4.

SEBESTA, R. W.; **Conceitos de linguagens de programação**. 9. ed. Porto Alegre: Bookman, 2011. Capítulo 9.